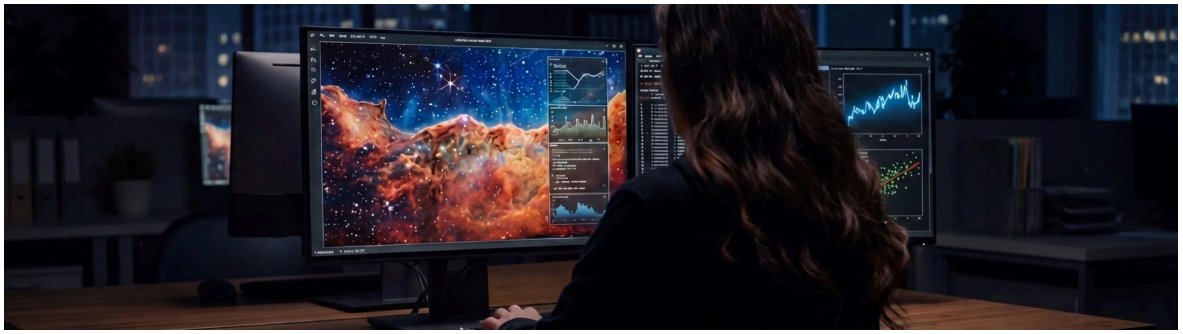


Google Summer of Code Proposal

Spectral timing in Julia

Contact Information

Field	Detail
Name	Alisha
Email	Alisha Patel
University	Scaler School of Technology (2nd Year)
Time Zone	IST (GMT + 5:30)
GitHub Profile	Alisha1701

Project Details

Project Title: Spectral timing in Julia

Mentors: fjebaker,matteobachetti

Organization: StingraySoftware/Stingray.jl

INDEX

- **Why I am a Good Fit for This Project**
- **Work Approach and Commitment**
- **Resources Used**
- **Use of LLMs (Large Language Models)**
- **Final Note on Passion**
- **Current State of Stingray.jl**
- **What's Missing (Critical for This Project)**
- **Community Bonding Period**
- **Proposed Work**
    a. 9.1 Phase 1: Core Spectral Timing API (Weeks 1–6)
        i. 9.1.1 Enhanced FITS I/O (`src/io.jl`)
        ii. 9.1.2 Test Infrastructure (TDD)
        iii. 9.1.3 Powerspectrum & Crossspectrum Types (`src/crossspectrum.jl`)
        iv. 9.1.4 Time Lags & Coherence Spectra
        v. 9.1.5 First Evaluation Deliverables
    b. 9.2 Phase 2: Variability-Energy Framework (Weeks 7–12)
        i. 9.2.1 Extended EventList / LightCurve Operations
        ii. 9.2.2 Variability-Energy Spectra (`src/varenergyspectrum.jl`)
        iii. 9.2.3 Power Colors (`src/powercolors.jl`)
        iv. 9.2.4 Covariance Spectra
        v. 9.2.5 Integration Testing & Documentation
        vi. 9.2.6 Final Evaluation Deliverables
- **Design Principles**
- **Timeline**
- **Stretch Goals**
- **Risk Mitigation**
- **Community Commitment**
- **My Recent Contributions (Pull Requests)**

- **Other:**

Why I am a Good Fit for This Project

I understand that I have not yet produced a significant volume of work. However, I have adhered to the organization's rules, specifically the one to "avoid the noise." Instead, I have focused on:

- **Learning Julia:** I have been learning the Julia language, understanding its benefits, and identifying common errors.
- **Data Comprehension:** I have spent time understanding the data coming from FITS files. Although I am still working on reading the FITS files, I have grasped basic column terminology like `TIME`, `DEADTIME`, and `MPU_A_TEMP`.
- **Conceptual Understanding:** Instead of blindly using LLMs, I have tried to understand the core terminology, such as what GTIs (Good Time Intervals) and BTIs (Bad Time Intervals) are, how they are derived from the data, and how plotting is performed. I am still trying to delve deeper into these concepts.

My current understanding of the processing chain is:

1. Load **event list** from a FITS file.
2. Convert to a **light curve** by binning the photon times.
3. Pass the light curve into a Fourier function like `powerspectrum()`.
4. Obtain a **periodogram object** with `freq` and `power` arrays.

I have also worked to understand how the Fourier transform works. Furthermore, I recognize the importance of shifting from Python to Julia. While Python is capable, there is a clear need to move to Julia due to the language's benefits like
Speed: Julia solves the "Two-Language Problem" by running high-level code at native C-speed without needing C-extensions.
Performance: The **JIT Compiler** turns your math loops into optimized machine code specifically for your IdeaPad's CPU.
Multiple Dispatch: Functions are independent of classes and choose the fastest "version" based on all input types (like your EventList recipes).
Composability: Different libraries (like DataFrames and FFTW) "just work" together automatically because they share the same type foundations.
Package Management: Pkg creates exact, reproducible environments that work perfectly on any machine.

Honestly, generating code is now often just one prompt away, and many people are doing that. However, I am not here merely to generate code; I am here to think and work like an astronomer.

I am still encountering some hurdles, but I am committed to the process.

Work Approach and Commitment

A key challenge for mentors is reviewing very large Pull Requests (PRs). Therefore, I have decided to submit all my PRs in an **atomic PR** fashion. This will help the mentors review small, manageable chunks of code. I noticed that last year, a significant amount of work could not be reviewed due to this very issue.

One goal of mine with this organization is to **work less but to work with precision, meaning, and robustness.**

I am watching videos and actively learning to gain in-depth knowledge of the domain.

I have always been space-obsessed and wanted to study aerospace but could not pursue it. This opportunity allows me to work in my field of interest, and I am excited to discover more in this area.

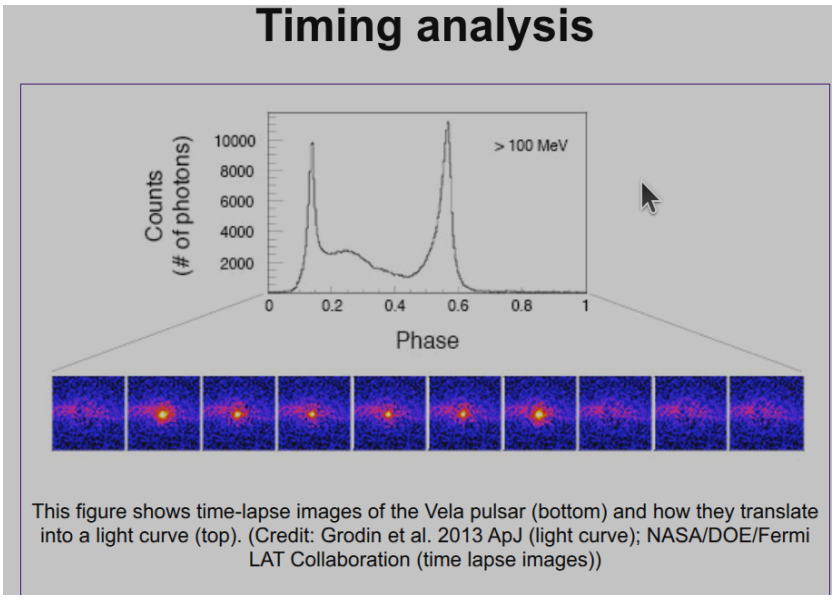
I learn slowly, and I make mistakes, but I am gradually building confidence and know that I can succeed with help, guidance, and continuous learning.

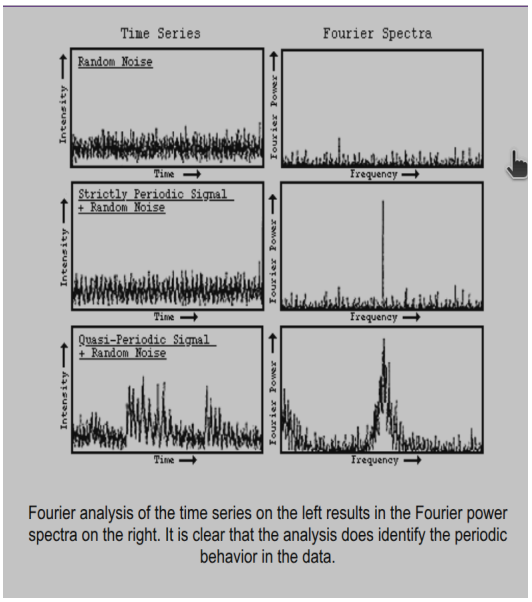
With that said, I want to thank last year's contributor, Kashish2210, for all the help and guidance he has provided.

Resources Used

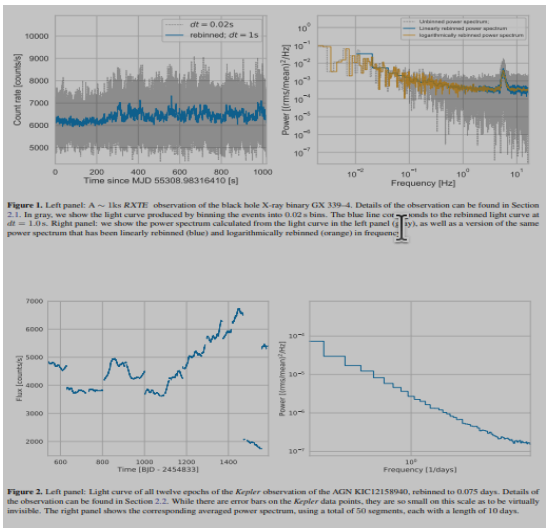
These are the resources I have used to gain knowledge in this domain. I am still learning.

1. [Timing analysis]





2. [The Stingray Paper] : To grasp the big picture of Stingray and its requirements. Will Reread for clarity.



3.Uttley et al.2014 : X-ray reverberation around accreting black holes
4.Heil, Uttley & Klein-Wolt 2015: Power colors methodology
5.Bendat & Piersol (2011) — *Random Data* : Random Data: Analysis and Measurement Procedures
6.van der Klis 1989 — Fourier techniques : teaches the fundamental methodological framework for applying Fourier analysis to high-energy astronomical time-series data (like X-ray event lists) to study variability.
7.FITSIO.jl docs : Reading/writing FITS files — essential for `io.jl`
8.RecipesBase.jl : Writing `@recipe` plot functions
9.FFTW.jl : Understanding FFT calls in fourier.jl
10.Julia Manual — Types, Methods, Interfaces : Understanding Julia-specific concepts.

Use of LLMs (Large Language Models)

I have used LLMs responsibly in the following ways:

- Explaining concepts
- Reading/explaining existing code
- Debugging
- API guidance
- Architecture advice
- Reviewing code
- Python Stingray reference
- Proposal writing

Final Note on Passion

I truly love this learning phase. I love this stars and galaxy analysis. It feels like I've finally gotten to do what I've always wanted, and it would be incredible if I got the chance to work more and more on this—something I've always wanted to do.

Current State of Stingray.jl

Module	Status	Key Functions
`events.jl`	Core complete	`readevents`, `filter_time!`, `filter_energy!`, FITS read

`lightcurve.jl`	Core complete	`create_lightcurve`, `bin_events`, `rebin`, error calculation
`gti.jl`	Near parity	`check_gtis`, `create_gti_mask`, `intersect`/`union`, `split_by_gtis`
`fourier.jl`	Core functions	Normalizations (Leahy/frac/abs), `avg_pds_from_events`, `avg_cs_from_events`, `raw_coherence`, `cross_to_covariance`
`utils.jl`	Utilities	`contiguous_regions`, `merge_overlapping_gtis`
`plotting`	Basic	`LightCurve` plot recipes via `RecipesBase.jl`

Test suite: 662 tests across 5 test files — all passing.

What's Missing (Critical for This Project)

Gap	Python Module	Priority
`Powerspectrum` / `Crossspectrum` types	`crossspectrum.py`, `powerspectrum.py`	Critical
Time lag extraction	`crossspectrum.py` → `time_lag()`	Critical
Coherence spectra (high-level API)	`crossspectrum.py` → `coherence()`	Critical
Variability-energy spectra	`varenergyspectrum.py`	Critical
Power Colors	`power_colors.py`	Critical
FITS I/O (write/export)	`io.py`	Important

Extended `EventList` operations	`events.py`	Nice-to-have

Community Bonding Period (≈ 3 weeks before coding)

1. Codebase familiarization: Deep-dive into Python Stingray architecture (base.py → crossspectrum.py → varenergyspectrum.py)
2. Simulated data validation: Generate synthetic X-ray event lists, run existing Julia functions, validate against Python outputs
3. Deliverable: Pluto notebook showing Events → LightCurve → PDS → Plot workflow
4. Mentor coordination: Align on API design, coding conventions, and review cadence

Proposed Work

The project is organized into two phases, aligned with the GSoC evaluation milestones.

Phase 1: Core Spectral Timing API (Weeks 1–6)

1.1 — Enhanced FITS I/O (`src/io.jl`) — Weeks 1–2

Function	Description
`write_eventlist(filepath, ev)`	Write `EventList` to FITS with proper HDU structure
`write_lightcurve(filepath, lc)`	Write `LightCurve` with TIME, RATE, ERROR columns
`write_crossspectrum(filepath, cs)`	Write cross spectrum results to FITS
`read_lightcurve(filepath)`	Read light curve from FITS

Design: Implement bidirectional FITS I/O following OGIP standards. Use `FITSIO.jl` (already a dependency), follow OGIP column naming conventions, and ensure round-trip fidelity (write → read → validate identical).

1.2 — Test Infrastructure (TDD) — Week 2

Write comprehensive test specifications before implementation:

```
# test/test_crossspectrum.jl
@testset "Powerspectrum" begin
   @test pds.freq ≈ expected_freq
   @test pds.power ≈ expected_power  atol=tolerance
   @test pds.norm == "frac"
   @test mean(pds.power[high_freq]) ≈ poisson_level("frac", meanrate=rate)
atol=...
end
@testset "Crossspectrum" begin
   @test cs.time_lag ≈ known_lag  atol=...
   @test cs.coherence ≈ 1.0  atol=...  # for correlated signals
End
```

1.3 — Powerspectrum & Crossspectrum Types (`src/crossspectrum.jl`) — Weeks 3–4

Implement Julia-idiomatic struct-based types with multiple dispatch constructors:

```
abstract type AbstractSpectrum end
struct Powerspectrum <: AbstractSpectrum
   freq::Vector{Float64}
   power::Vector{Float64}
   unnorm_power::Vector{Float64}
   n::Int                # bins per segment
   m::Int                # number of averaged segments
   dt::Float64           # time resolution
   df::Float64           # frequency resolution
   norm::String          # "leahy", "frac", "abs", "none"
   nphots::Float64
   gti::Matrix{Float64}
   segment_size::Float64
   # ...
end
struct Crossspectrum <: AbstractSpectrum
   freq::Vector{Float64}
   power::Vector{ComplexF64}
   unnorm_power::Vector{ComplexF64}
```

```
    pds1::Union{Nothing, Vector{Float64}}    # auxiliary PDS
    pds2::Union{Nothing, Vector{Float64}}
    n::Int
    m::Int
    dt::Float64
    df::Float64
    norm::String
    nphots1::Float64
    nphots2::Float64
    gti::Matrix{Float64}
    segment_size::Float64
    # ...
end
# Constructor from EventList (wraps existing avg_pds_from_events)
function Powerspectrum(ev::EventList, segment_size, dt; norm="frac")
    # internally calls avg_pds_from_events
end
# Constructor from two EventLists
function Crossspectrum(ev1::EventList, ev2::EventList, segment_size, dt;
norm="frac")
    # internally calls avg_cs_from_events
End
```

Key Design: These types wrap existing Fourier functions from `fourier.jl` rather than reimplementing them, ensuring consistency and reuse.

1.4 — Time Lags & Coherence Spectra — Weeks 5–6

Methods on `Crossspectrum` that leverage the existing Fourier layer:

```
function time_lag(cs::Crossspectrum)
    phase = angle.(cs.power)
    lag = phase ./ (2π .* cs.freq)
    # Error: Bendat & Piersol (2011) formula
    coh = coherence(cs)
    lag_err = sqrt.((1 .- coh) ./ (2 .* coh .* cs.m)) ./ (2π .* cs.freq)
    return lag, lag_err
end
function coherence(cs::Crossspectrum)
    # Uses existing raw_coherence from fourier.jl
    coh = raw_coherence.(cs.power, cs.pds1, cs.pds2, noise1, noise2, cs.m)
    return coh, coh_err
End
```

Plus plotting recipes for the new spectral products.

```julia
@recipe function f(pds::Powerspectrum)
    xscale --> :log10
    yscale --> :log10
    xlabel --> "Frequency (Hz)"
    ylabel --> "Power"
    pds.freq, real.(pds.power)
end
```

1.5–First Evaluation Deliverables (Week 6)

- FITS I/O read + write (`io.jl`)
- `Powerspectrum` and `Crossspectrum` types with constructors
- Time lag and coherence extraction
- Plotting recipes for spectral products
- Test suite: 662 existing + ~150 new tests, all passing

Phase 2: Variability-Energy Framework (Weeks 7–12)

2.1 — Extended EventList / LightCurve Operations — Week 7

Function	Description
`join(ev1, ev2)`	Concatenate `EventLists`, merging GTIs
`sort!(ev)`	Sort events by time in-place
`filter_energy_range(ev, emin, emax)`	Convenience energy filtering
Arithmetic ops (+, -) on LCs	Add/subtract light curves
`truncate(lc, tmin, tmax)`	Slice light curve by time range

2.2 — Variability-Energy Spectra (`src/varenergyspectrum.jl`) — Weeks 8–10

This is the most scientifically significant deliverable, implementing how X-ray variability changes as a function of photon energy.

```
abstract type AbstractVarEnergySpectrum end
struct VarEnergySpectrumConfig
   freq_interval::Tuple{Float64, Float64}     # [f_low, f_high]
   energy_bins::Vector{Float64}               # energy bin edges
   ref_band::Matrix{Float64}                  # reference band
   bin_time::Float64
   segment_size::Float64
   use_pi::Bool
end
struct LagEnergySpectrum <: AbstractVarEnergySpectrum
   config::VarEnergySpectrumConfig
   energy::Vector{Float64}
   spectrum::Vector{Float64}
   spectrum_error::Vector{Float64}
end
struct RmsEnergySpectrum <: AbstractVarEnergySpectrum ... end
struct CovarianceSpectrum <: AbstractVarEnergySpectrum ... end
```

Core algorithm (shared across all variability spectra):

```
For each energy bin [E_low, E_high]:
 1. Filter events → subject band (this energy bin)
 2. Filter events → reference band (all energies MINUS this bin)
 3. Compute cross spectrum between subject and reference light curves
 4. Average over segments
 5. Select frequency range [f_low, f_high]
 6. Compute the specific quantity:
    - Lag:        arg(⟨C⟩) / (2π⟨f⟩)
    - RMS:        √(Σ|C|² Δf) / ⟨ref_rate⟩
    - Covariance: cross_to_covariance(⟨C⟩, ⟨P_ref⟩, noise, Δf)
```

2.3 — Power Colors (`src/powercolors.jl`) — Week 10

Implement the model-independent variability classification method from Heil, Uttley & Klein-Wolt (2015):

```
struct PowerColor
   pc1::Float64          # variance ratio: band C / band A
```

```
    pc2::Float64            # variance ratio: band D / band B
    pc1_error::Float64
    pc2_error::Float64
    hue::Float64            # angular position on the color-color diagram
    hue_error::Float64
end
function power_colors(pds::Powerspectrum;
                      freq_edges=[1/256, 1/32, 0.25, 2.0, 16.0])
    # Split power spectrum into 4 frequency bands (A, B, C, D)
    # PC1 = variance(C) / variance(A)
    # PC2 = variance(D) / variance(B)
    # Hue = atan2(PC2 - PC2_ref, PC1 - PC1_ref)
    return PowerColor(pc1, pc2, pc1_err, pc2_err, hue, hue_err)
end
# Plotting: color-color diagram
@recipe function f(pcs::Vector{PowerColor})
    xlabel --> "PC1 (Hard color)"
    ylabel --> "PC2 (Soft color)"
    [pc.pc1 for pc in pcs], [pc.pc2 for pc in pcs]
End
```

Power colors provide a model-independent way to classify X-ray source states—tracking how a source moves through hard/soft states on a 2D color-color diagram without requiring Lorentzian model fitting. The hue value encodes the angular position, enabling automated state classification.

2.4 — Covariance Spectra — Week 11

Builds on the existing `cross_to_covariance` function in [fourier.jl]():

```
function CovarianceSpectrum(ev::EventList; freq_interval, energy_spec,
                            ref_band=nothing, segment_size, bin_time)
    config = VarEnergySpectrumConfig(...)
    for (i, (elow, ehigh)) in enumerate(energy_intervals)
        # Compute averaged cross spectrum → apply cross_to_covariance
    end
End
```

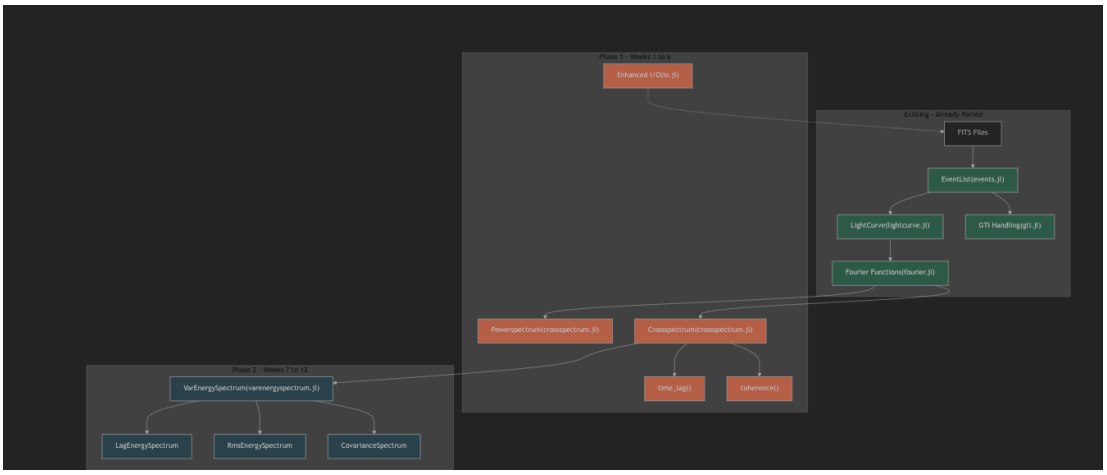
2.5 — Integration Testing & Documentation — Week 12

- End-to-end pipeline: FITS → `EventList` → `Crossspectrum` → `LagEnergySpectrum` → Plot.

- Cross-validation: Compare outputs against Python Stingray on identical datasets.
- Documentation: Docstrings for all public functions + usage examples.
- Demo notebook: Pluto.jl notebook demonstrating the complete workflow.

2.6—Final Evaluation Deliverables (Week 12)

- `LagEnergySpectrum`, `RmsEnergySpectrum`, `CovarianceSpectrum`
- Power Colors with hue computation and color-color diagram plotting
- Extended `EventList` / `LightCurve` operations
- Complete FITS I/O for all data products
- Plotting recipes for all spectral types
- Comprehensive test suite (target: $\geq$900 tests)
- Cross-validated against Python Stingray
- Documentation with examples and a demo notebook



Design Principles

Principle	Implementation
Julia idioms	Multiple dispatch + parametric types, not OOP inheritance
Build on existing layer	New types wrap avg_pds_from_events / avg_cs_from_events
Test-driven development	Test specs written *before* implementation
Cross-validation	Every result must match Python Stingray on identical inputs

Performance by design	@views, in-place ops, FFTW planning for hot loops
Composability	Functions dispatch on abstract types for extensibility

Timeline

Week	Phase	Deliverable
Pre-coding	Community Bonding	Codebase deep-dive, simulated data validation, CI/CD setup
1–2	Phase 1	Enhanced FITS I/O (io.jl), test infrastructure (TDD)
3–4	Phase 1	Powerspectrum & Crossspectrum types with constructors
5–6	Phase 1	Time lags, coherence, plotting recipes
6	1st Evaluation	Core spectral timing API complete
7	Phase 2	Extended EventList/LightCurve operations
8–9	Phase 2	Variability-energy spectra (Lag, RMS)
10	Phase 2	Power Colors (powercolors.jl) — hue, color-color diagrams
11	Phase 2	Covariance spectra refinement + edge cases
12	Phase 2	Integration testing, cross-validation, documentation
12	Final Evaluation	Complete variability-energy framework

Stretch Goals

If the main deliverables are completed ahead of schedule:

1. Logarithmic rebinning — rebin_log for Powerspectrum/Crossspectrum (essential for publication-quality plots)
2. Frequency-resolved spectroscopy — initial framework for spectral decomposition by frequency
3. Benchmark suite — automated performance comparison between Julia and Python Stingray on standard datasets
4. Mission-specific helpers — basic support for NICER and NuSTAR data formats
5. Automated state classification — use power color hue values to automatically classify source states (hard, intermediate, soft)

Risk Mitigation

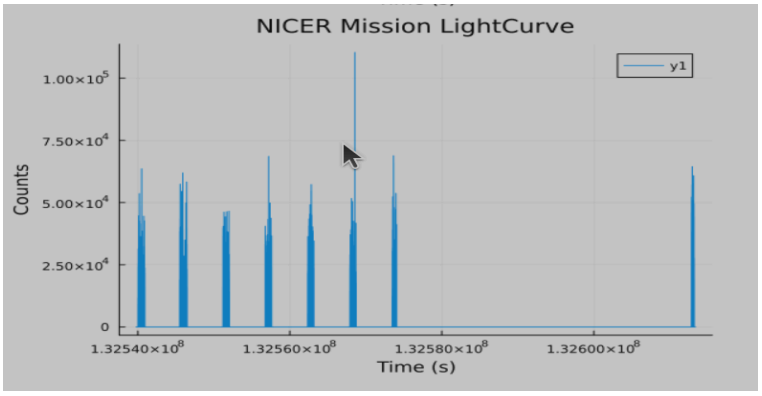
Risk	Strategy
Numerical precision differences (Python vs Julia)	Use atol/rtol tolerances in cross-validation; document any differences
Complex GTI handling in variability spectra	GTI module is already near-parity — leverage existing infrastructure
Scope creep	Explicitly exclude model fitting and simulation — focus on analysis pipeline
Large FITS file performance	Memory-mapped arrays where possible; benchmark with real mission data
Julia version compatibility	Pin minimum Julia version; test on current LTS and stable

Community Commitment

Even after the gsoc period is finished, I will stay connected, continue exploring opportunities within similar organizations, and strive to help the community.

My Recent Contributions (Pull Requests)

- StingraySoftware/Stingray.jl(open PR)
  1.Refactor plotting recipes and add comprehensive test suite for Issue #29 #72



- StingraySoftware/notebooks (merged)
  1.Fix #123: Correct ntrial calculation in Pulsar search notebook #125
- OpenAstronomy/openastronomy.github.io
  1. Sort members list alphabetically #397 (merged)
  2.Improve visibility of chat links in student guide #399 (merged)

My work can be seen here : My notebook

Other:

I have not participated in any GSOC project before, this is my first time.
I am applying to one more org under openastronomy only Spectra.jl across the electromagnetic spectrum
I will be available during the GSOC coding period and I will handle the academic load by proactively communicating with my college teachers and mentors to manage my schedule and commitments effectively during the program.
This is my other GitHub acc for projects: Alisha-2005